

参考文献

- [1] NYMAN L, LAAKSO M. Notes on the history of fork and join [J]. IEEE Annals of the History of Computing, 2016, 38 (3): 84-87.
- [2] RITCHIE D M, THOMPSON K. The unix time-sharing system [J]. Bell System Technical Journal, 1978, 57 (6): 1905-1929.
- [3] RITCHIE D M. The evolution of the unix time-sharing system [C] // Symposium on Language Design and Programming Methodology. 1979: 25-35.
- [4] BAUMANN A, APPAVOO J, KRIEGER O, et al. A fork() in the road [C] // Proceedings of the Workshop on Hot Topics in Operating Systems. 2019: 14-22.

进程与线程：扫码反馈



处理器调度

现代操作系统支持多个进程同时运行，为了协调进程对于有限处理器资源的使用，操作系统引入了处理器调度（CPU Scheduling）。通过管理多个进程的执行，调度器尽可能保证程序运行满足系统和应用的预定指标，例如高吞吐量、低时延或低功耗。操作系统诞生至今，对于处理器调度的研究从未中断，这也体现了处理器调度对于操作系统的重要性。本章首先针对单个处理器核心的场景，围绕处理器调度，从机制、指标、策略三个方面介绍调度器的设计与实现，之后简要介绍多核处理器场景下的调度。此外，本章会对现代操作系统中的调度器实例进行分析，并讨论处理器调度在现代多处理器场景中所需要面临的挑战。在阅读本章后，相信读者会对处理器调度形成自己的理解，并能够回答以下问题：

- 处理器调度机制有哪些？系统中存在大量进程或线程，操作系统如何调度这些进程或线程？
- 处理器调度指标有哪些？调度器的目的是让多个进程能够更“好”地共享处理器。为了评价调度器，有哪些常见的调度指标？
- 处理器调度策略有哪些？针对不同的工作场景，主要的调度指标也不尽相同。为了满足多变的调度指标，开发者需要有针对性地选择调度策略，那么有哪些常见的处理器调度策略？

7.1 处理器调度机制

本节主要知识点

- 任务在操作系统中是如何被调度的？
- 什么是空闲进程？为什么要有空闲进程？

处理器调度的设计遵循“将策略与机制分离”的原则。调度机制包括调度对象的设

置、调度接口的设计、调度时机的选择以及不同任务的切换方法。在调度机制的基础上，操作系统可以支持不同的调度策略，供系统管理员灵活选择。本节将主要介绍单处理器核心的调度机制，与多核处理器调度机制相关的内容会在 7.6 节进行简要介绍。

7.1.1 处理器调度对象

处理器调度的对象是 CPU 执行的最小单元。在操作系统发展的早期，每个程序在运行时对应操作系统中的一个进程。当时进程是程序执行的最小单元，所以进程也成为当时处理器调度的对象。随着硬件的发展和操作系统的演进，出现了线程的抽象与多线程机制。此后，处理器调度的对象由进程变为线程，早期的进程则被看作单线程的进程。从原理上讲，处理器调度的对象是线程，但对于本节的内容来说，线程与进程并没有太大的区别。为了避免对处理器调度对象这一概念的混淆，本章将统一用任务（Task）来描述处理器调度的对象。在不同的系统中，任务可以是进程或线程。

7.1.2 处理器调度概览

处理器调度一般分为两步：第一步选取需要调度的任务，第二步在目标 CPU 核心上执行该任务。其中，第二步是通过 6.4 节介绍的上下文切换完成的。那么，操作系统如何指定下一个需要调度的任务呢？图 7.1 给出了处理器调度的概览。

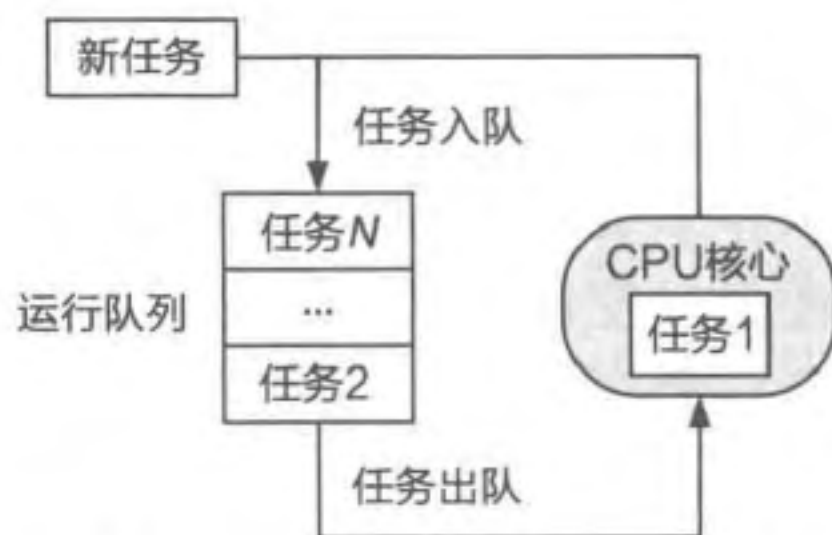


图 7.1 处理器调度的简略示意图

运行队列

假设在系统中一共有 N 个任务，其中任务 1 正在 CPU 核心上执行，其余任务被记录于运行队列^①（Run Queue）中，等待被调度并执行。由于操作系统以运行队列为抽象管理任务，所以调度的具体操作可以抽象为任务的入队（Enqueue）与出队（Dequeue）。需要注意的是，运行队列并非一定是先进先出的队列，而可以是任何一种适合当前调度策略的数据结构。例如，Linux 的完全公平调度器使用红黑树维护有序的运行队列，使插入任务的时间复杂度为 $O(\log N)$ ，选择下一个任务的时间复杂度为 $O(1)$ （后文会进一步介绍）。

调度接口

代码片段 7.1 展示了相应的调度框架接口，即与调度相关的主要函数^②，与图 7.1 相对应。每当操作系统调用一次 sched，当前 CPU 核心便会触发一次调度。具体地，系统会通过 sched_enqueue 将当前执行的任务（任务 1）重新放回运行队列，再通过

① 运行队列也常被称为就绪队列（Ready Queue），这里的“运行”并非指进程处于正在运行的状态，而是指任务处于可以被运行的状态。

② 代码中仍使用 process 进程作为参数，表示调度对象。

`sched_dequeue` 选取下一个需要执行的任务（任务 2）。最后，系统设置相关的参数，确保当前 CPU 核心从内核态返回用户态时使用任务 2 的处理器上下文，而非任务 1 的处理器上下文，从而达到任务切换的效果。当一个新的任务被创建时，系统也会通过调用 `sched_enqueue` 将其添加进运行队列中。

代码片段 7.1 操作系统中的主要调度接口

```
1 struct sched_ops {
2     // 触发一次在当前 CPU 核心上的调度
3     int (*sched)(void);
4     // 将所给调度对象置于运行队列进行管理的辅助函数
5     int (*sched_enqueue)(struct process* proc);
6     // 从运行队列中选取下一个执行对象的辅助函数
7     struct process* (*sched_dequeue)(void);
8 };
```

调度时机

在什么情况下系统会触发调度呢？这就引入了调度时机这一概念，即系统触发处理器调度的时机。理论上，可以认为只要系统进入内核态，就有可能触发调度。具体来说，操作系统会在内核态代码的一些位置插入对 `sched` 函数的调用，当运行到这些位置时，便会触发调度。触发调度后，当处理器从内核态返回用户态时，便会切换到下一个需要执行的任务的上下文。具体在哪些位置插入 `sched` 函数，会因操作系统设计的差异而有所区别，但总体上可以根据是不是“当前执行的任务主动发起”这一条件，分为协作式 (Cooperative)[又称非抢占式 (Non-Preemptive)] 与抢占式 (Preemptive) 两大类。

协作式调度的时机一般是当前任务执行结束时或当前任务主动放弃执行机会时。第一种情况相对易于理解，我们主要讨论第二种。操作系统一般会提供类似名为 `yield` 的系统调用，其主要逻辑就是在内核态调用 `sched`。用户态的任务可通过主动调用 `yield` 陷入内核态并触发处理器调度，让系统选择其他任务执行。此外，当任务发起阻塞式 I/O 请求后[⊖]，在等待 I/O 结果的这段时间内无法继续执行。因此，包括 ChCore 在内的许多操作系统会在上述这类系统调用的最后调用 `sched` 以触发调度，使当前 CPU 核心切换并执行其他任务。

那么，如果当前任务并不主动执行 `yield` 或阻塞式 I/O，系统该如何触发调度呢？此时就需要抢占式调度：通过中断机制，操作系统可以强制打断当前任务的执行，从而有机会触发调度。操作系统一般将定时触发的时钟作为稳定的中断来源，让系统能够定时抢占任务的执行，通过在时钟中断处理函数的末尾插入 `sched` 调用，就能触发处理器调度。此外，大部分操作系统会为任务指定时间片 (Time Slice)，表示该任务可以执行的时间长度。当处理时钟中断时，会检查当前任务的已执行时间是否超过了被分配的时间片，如果超过则触发调度。

⊖ 阻塞式 I/O 的一个例子是 Shell 等待用户的键盘输入。

调度策略

在到达调度时机后，处理器调度器根据具体的调度策略做出调度决策，然后系统根据该决策进行任务切换。这里的调度决策包括：①从运行队列中选择下一个运行的任务；②决定该任务下一次允许运行的时间，即时间片。根据调度策略的不同，在相同情况下做出的决策也会不尽相同。回顾代码片段 7.1 中的调度相关函数，它们实际上是操作系统的处理器调度所提供的接口。操作系统可以通过为这些接口提供不同的实现，允许用户选择不同的调度策略，达到机制与策略分离的效果。

在针对处理器调度的研究中，对于调度策略的研究一直是主旋律。这是因为没有一种完美的调度策略可以适应所有的场景。在设计调度策略的时候，需要考虑以下两个问题。

- 第一个问题是应该选择什么样的调度指标（What）？操作系统使用调度器的目的是通过协调任务对 CPU 的使用，进而使任务的执行在某一方面达到用户的预期，或者说达到用户指定的调度指标。那么有哪些指标？在不同种类的系统中应该考虑何种指标？这些都是开发者在设计实现调度器时需要考虑的因素，具体会在 7.2 节进行详细介绍。
- 第二个问题是应该如何选择符合预期的调度决策（How）？在确定当前系统需要考虑的调度指标后，我们需要进一步通过适当的调度策略做出合理的决策。应该如何选择合理的调度策略？这需要开发者对于不同调度策略有着清晰的认识，具体会在 7.3 ~ 7.5 节进行详细介绍。

小知识：空闲进程

细心的读者可能会问，如果运行队列中没有可执行的任务，那么 CPU 核心应该执行什么代码呢？实际上，当运行队列为空时，CPU 核心一般会执行一段预设的“无意义”循环代码。为了处理器调度设计的统一性，大部分操作系统都倾向于让上述“无意义”的代码由一类特殊的进程执行，此类进程被称为**空闲进程（Idle Process）**。这段“无意义”的代码一般会让 CPU 核心进入低功耗模式。以 ChCore 为例，在 ARM 架构下，空闲进程会执行 wfi 指令^①，直到中断发生，当前 CPU 才会重新正常工作。

练习

7.1 既然低功耗模式可以节省能耗，那么操作系统是否应该让 CPU 核心一有机会就进入低功耗模式呢？

① 该指令全名为 wait for interrupt。执行该指令的 CPU 会停止执行代码并进入低功耗模式，直到该 CPU 接收到中断为止。

7.2 处理器调度指标

本节主要知识点

- 常见调度指标有哪些？
- 互相冲突的调度指标有哪些？

本节将介绍处理器调度指标，这些指标被用于评判调度策略的优劣，具体的调度策略会在后续章节讲解。需要注意的是，在评判一个调度策略的优劣时，一定是评判该调度策略在特定场景下的优劣。这是因为计算机的应用场景复杂多变，用户对于不同场景下的任务执行会有不同的预期，因此会有不同的调度指标来指导调度策略的选择。本节将介绍下列主要的调度指标：平均周转时间、平均响应时间、实时性、资源利用率、公平性和调度机制的开销。

平均周转时间

在计算机处理的任务中，有一类任务被称为批处理任务，比如机器学习的训练、复杂的科学计算等，它们在计算机上执行时无须与用户交互，其目标就是尽快完成。这类任务的主要调度指标是周转时间（Turnaround Time），即任务在系统中总共花费的时间。由于任务只要在系统中就会消耗资源，所以任务的周转时间越小越好。其计算公式是任务的完成时间点（Completion Time）与到达时间点（Arrival Time）之差^①：

$$T_{\text{周转}} = t_{\text{完成}} - t_{\text{到达}}$$

其中，到达时间点是任务在系统中被创建并开始等待处理器执行的时间点，完成时间点是任务结束执行并在系统中被销毁的时间点。

实际上，一个任务的周转时间还是它的等待时间（Waiting Time）与运行时间（Burst Time）之和：

$$T_{\text{周转}} = T_{\text{等待}} + T_{\text{运行}}$$

其中，等待时间是任务在系统中等待被执行所花费的时间，运行时间是执行任务所需的时间。一个任务的运行时间越长，所消耗的处理器的资源越多。根据运行时间的相对长短，任务可以被称为长任务或短任务。

在某些情况下，任务的运行时间是大致不变的，因此任务的周转时间越长，则代表任务花费了越多的时间在等待执行上。系统中所有任务的平均周转时间是一个经典的调度指标，用于衡量任务在系统中等待执行的情况。该指标越小，代表系统中全部任务花费在等待上的时间越少。

平均响应时间

随着计算机的应用场景越来越多，人们开始需要使用计算机执行一些交互式任

① 在本章的公式中， T 表示时间段， t 表示时间点。